

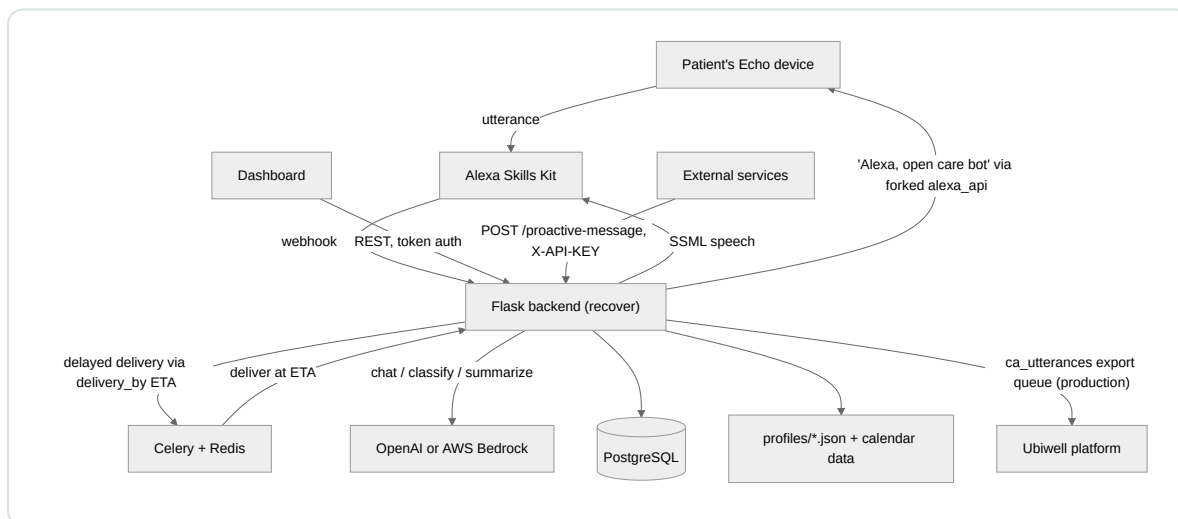
Robin Database Review

RECOVER study conversational agent · reviewed 2026-09-08 · companion
to the [ER diagram](#)

Summary. Robin is an Alexa-based AI companion for elderly residents of an assisted-living facility, paired with a dashboard for clinicians. The backend is Flask + SQLAlchemy on PostgreSQL (migrated from SQLite), with Celery for scheduled proactive messages. The repository has two meaningfully different lines of development: the deployed `production` branch is a restructured, tested codebase with voice-speaker recognition, delivery telemetry, and a Ubiwell export pipeline — and no vector store — while `master` is an older monolith that still carries ChromaDB retrieval and ~32 dynamically generated symptom columns. One security issue is present on *both* branches and worth fixing promptly: patient API responses leak clinician password hashes and Alexa refresh tokens. The remaining findings are integrity gaps that are manageable at a 15-participant scale.

1. System context

Everything routes through one Flask app (`backend/recover/`). Patients talk to an Echo device; a catch-all Alexa handler treats each utterance as free text and drives a GPT-4.1 (or Bedrock) conversation loop. Clinicians use the dashboard over token-authenticated REST. External services push health events in through a service-key API, which Robin turns into spoken proactive check-ins. On production, an export worker also streams conversation utterances out to the Ubiwell research platform.



Request and data flow on the production branch. Proactive delivery works by literally commanding the device to open the skill, then serving the prepared message as the "last message" when the skill launches. On master, a ChromaDB vector store also sits beside Postgres for conversation retrieval.

2. Two branches, two architectures

The `production` branch (tip: "Prod agentic changes", PR #51) is not just ahead of `master` — it is a restructure: 139 files changed, with the monolithic `utils.py` / `vectordb.py` / `proactive_message.py` split into `services/` (alexa session lifecycle, conversation engine and flow, reminders), `skills/` (check-ins), `intent/` (an intent classifier), `llm/`, `proactive/`, and `prompt_context/` packages — plus a `backend/tests/` suite that `master` lacks entirely. Functionally, production:

- **Removed ChromaDB retrieval entirely** — no vector store, no query-classification RAG step; context now comes from intent handlers, profiles, and prompt context.
- **Added voice-speaker recognition** — Alexa person IDs are mapped to patients through a new `alexa_voice_profile` table, and each message records who was speaking.
- **Added delivery telemetry** — per-message status, correlation ID, HTTP status, latency, and error columns on `conversation_log`.
- **Added a Ubiwell export pipeline** — an in-process queue and worker thread POSTs conversation utterances to a research data platform, gated by a per-patient flag and a global `app_setting` toggle, with export status tracked per conversation.
- **Dropped the dynamic symptom columns** from `conversation` along with `symptoms.py`.

3. Storage topology

PostgreSQL is the system of record, joined by file-based stores that carry data the app depends on:

- **Profile JSON files** (`profiles/<external_id>.json`) — names, family, hobbies, tone preferences. This is a deliberate privacy design: patient identity never enters Postgres, which stays anonymized as participants P1–P15.
- **Calendar/schedule data** — fetched from a URL, cached to disk, and injected into the conversation prompt as today's schedule.
- **ChromaDB** (`./chroma_db`, master only) — chunks of every user utterance, embedded with `text-embedding-3-small` at session end and queried for retrieval during conversation.

The SQLite past is still visible: `backend/migrate_data.py` copies all tables to Postgres with FK filtering and `ON CONFLICT DO NOTHING`, and `fix_server_sequences.py` exists because the copy left auto-increment sequences stale.

4. Table reference (production)

Thirteen tables. The hub is `conversation` — one row per voice session — which everything on the dashboard hangs off.

TABLE	PURPOSE	NOTES
patient	Study participant + Alexa device identity	Holds <code>alexa_refresh_token</code> / <code>alexa_device_name</code> used to command the device; <code>external_id</code> keys into profile JSONs; production adds <code>ubiwell_metadata</code> , <code>location_coordinates</code> , and a per-patient export opt-in.

TABLE	PURPOSE	NOTES
user	Dashboard clinician account	Hashed password, <code>is_admin</code> . Many-to-many with patients, but in practice every user is attached to every patient.
user_patient_table	Join table	Auto-created patients call <code>users.extend(User.query.all())</code> .
alexa_voice_profile	Speaker recognition (production)	Maps <code>(alexa_user_id, alexa_person_id)</code> — unique together, both indexed — to a patient, with display name and profile key.
conversation	One voice session (was <code>report</code>)	<code>interaction_type</code> reactive/proactive, JSON <code>state</code> , plus Ubiwell export status/timestamps/error. On master this table instead carries ~32 symptom columns generated from <code>symptoms.py</code> .
conversation_log	Individual message	<code>require_affirmation</code> gates "do you have a minute?"; <code>redacted</code> supports the delete-message flow; production adds <code>channel</code> , <code>event_type</code> , six delivery-telemetry columns, and speaker identity.
report_summary	LLM-generated session summaries	Regenerated (delete-then-insert) at each session end; <code>conversation_log_ids</code> is a CSV string parsed client-side.
report_note	Clinician note on a session	Create and delete only, no edit — per the study design.
report_intervention	Clinician intervention record	Structurally identical to notes.
token	Dashboard auth token	Plain token string with <code>rememberme</code> flag.
proactive_message	Inbound event from external services	Unique on <code>(service, message_id)</code> for idempotency; severity 0–1, >0.7 delivers immediately; <code>delivered</code> flag drives context selection.
app_setting	Global key-value settings (production)	String key, JSON value — e.g. the global <code>ca_utterance_export_enabled</code> toggle.

TABLE	PURPOSE	NOTES
authorized_service	Service API key	Indexed <code>api_key</code> checked via the <code>X-API-KEY</code> header.

Plus `alexa_id_note` , a scratch table recording unknown Alexa IDs when a patient says "please note my Alexa ID".

Session boundary semantics

`get_or_create_conversation` cuts a new session when the last message contains the literal keyword `CONVERSATION_END` — which the LLM is prompted to append to goodbyes — or when the previous session is more than 10 minutes stale (`CONVERSATION_TIMEOUT_SECONDS`). Session lifecycle is therefore partly under the model's control: if the LLM forgets the keyword, sessions merge; if it emits it early, they split.

The dynamic symptom columns (master only)

On master, a loop over the `symptom_descriptions` dict calls `setattr(Conversation, ...)` at import time, producing `<symptom>_state` (0–4 color scale) and `<symptom>_logs` for 13 symptoms, plus `<symptom>_scale` for the six Likert-type ones — the dashboard's color-coded report table reads these directly. Production dropped the loop, the dict, and the columns.

5. Findings

SECURITY • BOTH BRANCHES • FIX PROMPTLY

Patient endpoints leak clinician password hashes and device tokens. `db.py` monkey-patches a generic `as_dict()` onto every model that also expands relationships in `__relationship_keys__` . `Patient` lists `users` , so the patient list, create, and detail endpoints return every clinician's password hash — and the patient's `alexa_refresh_token` — to any logged-in dashboard user (verified on master at `apis.py:194/366/450` and on production at `apis.py:236/352/435`). The code strips `password` when serializing users directly but not on this nested path. Fix: exclude sensitive columns in `as_dict()` itself rather than at call sites.

BUG • MASTER ONLY

Proactive context queries summaries by the wrong key. Master's `generate_enhanced_message` (`proactive_message.py:213`) runs `ReportSummary.query.filter_by(conversation_id=current_message["patient_id"])` — filtering a conversation ID column by a patient ID, returning another patient's summaries or none. Production replaced this function with a context-free message rewrite, eliminating the bug.

PRIVACY GAP • MASTER ONLY

Redaction does not reach ChromaDB. On master, the delete-message flow overwrites `conversation_log.content` with `"REDACTED"` , but chunks already embedded in the vector store are never removed, so deleted content can still surface through retrieval. Moot on production, which removed the vector store.

INTEGRITY · BOTH BRANCHES

`proactive_message.patient_id` has no foreign-key constraint — it is a plain `Integer`, unlike every other patient reference, so orphaned messages are possible and master's wrong-column bug went uncaught.

MIGRATION HYGIENE · BOTH BRANCHES

Rename residue and branched history. The `report` → `conversation` rename (June 2025) never renamed the Postgres sequence, so both migration scripts carry a `'conversation': 'report_id_seq'` override; `report_summary` / `report_note` / `report_intervention` kept the old prefix; most Alembic revisions are unnamed and one branch needed a merge-heads revision. Production's schema also now diverges from master's migration chain — reconciling them later will take care.

DURABILITY · BOTH BRANCHES

Background work runs on in-process threads. Session-end summarization runs on a fire-and-forget daemon thread (master), and production's Ubiwell exporter is an in-process queue and worker thread — a crash or restart mid-write loses work silently, and summaries are delete-then-reinserted with no transaction spanning both steps. Production does mitigate the exporter with per-conversation status columns (`queued/exported/failed`) that allow retry.

6. Overall assessment

For its purpose — a 15-participant research deployment with a small trusted dashboard audience — the design is pragmatic and mostly sound: idempotent message intake, an anonymization boundary that keeps names out of the database entirely, and on production a genuinely improved structure with tests, speaker attribution, and observable delivery. The one item to act on now is the nested-serialization leak, which ships on both branches and is a small fix in `as_dict()`. Beyond that, the main structural risk is the master/production divergence itself: two schemas, two architectures, and a migration chain that only production extends.

Sources: `backend/recover/db.py`, `apis.py`, `alexa.py`, `proactive_message.py` / `proactive/service.py`, `vectordb.py`, `services/conversation/{engine,exporter}.py`, `symptoms.py`, `migrate_data.py`, `fix_server_sequences.py`, READMEs. Branches: `production` @ d7b85b5, `master` @ 2e7e25e.

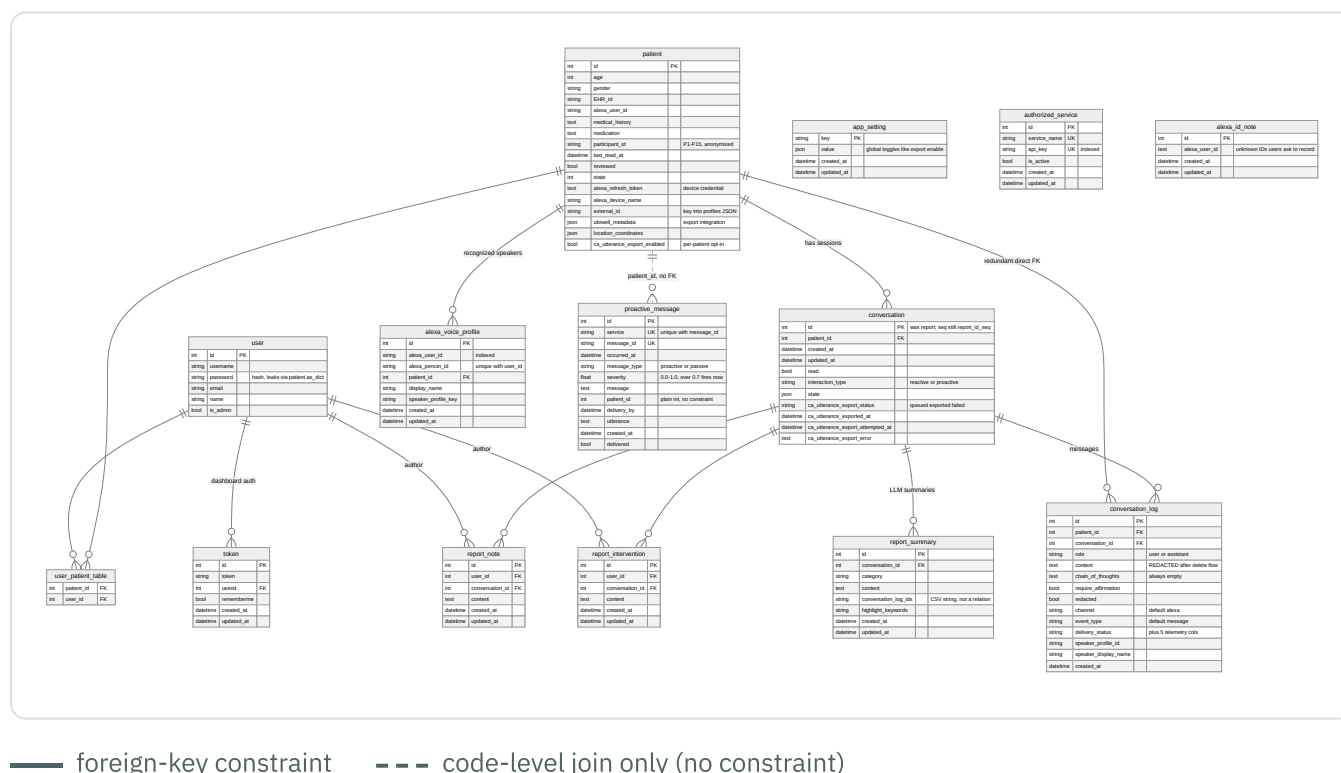
Robin Data Model

PostgreSQL schema for the RECOVER study's "Robin" conversational agent — Flask-SQLAlchemy models with Alembic migrations. Thirteen tables on the `production` branch: patients and their voice sessions on the left, the clinician dashboard on the right, service integration and export at the bottom.

Branch note. This diagram shows the deployed `production` branch. `master` differs in three ways: it still carries ~32 dynamically generated symptom columns on `conversation` (dropped in production), it lacks the `alexa_voice_profile` and `app_setting` tables plus the `delivery/speaker/export` columns, and it still uses a ChromaDB vector store for retrieval (removed in production).

Entity-relationship diagram

Crow's-foot notation. Solid lines are real foreign-key constraints; the dashed line to `proactive_message` is a join the code performs on a plain integer column with no constraint behind it.



The stores next to Postgres

The relational schema is only part of the picture — file-based stores hold data the app depends on, linked by keys Postgres knows nothing about enforcing.

Profile JSON files

`profiles/<external_id>.json`

Names, family, hobbies, communication preferences. Patient identity deliberately never enters Postgres — the DB stays anonymized (P1–P15) and identity lives only in these files.

Calendar / schedule data

`shared/data/calendar.json` + monthly cache

Schedule data fetched from a URL and cached to disk, injected into the conversation prompt as "today's schedule".

ChromaDB vector store (*master only*)

`./chroma_db · metadata.conversation_id`

On `master`, embedded chunks of every user utterance power RAG. Production removed the vector store entirely — retrieval was replaced by intent handlers and profile/context injection.

Constraint and schema quirks

Password leak path (both branches): the monkey-patched `as_dict()` expands `patient.users`, so patient endpoints return clinician password hashes and the patient's `alexa_refresh_token`.

Missing FK (both branches): `proactive_message.patient_id` is a plain `Integer`; every other patient reference is a real constraint.

Rename residue (both branches): `report` was renamed to `conversation` but its sequence is still `report_id_seq`, and `report_summary` / `report_note` / `report_intervention` kept the old prefix.

Stringly-typed relation (both branches): `report_summary.conversation_log_ids` is a comma-separated string parsed client-side for the dashboard's jump-to-log feature.

Session boundary is a keyword (both branches): a new `conversation` row is cut when the last message contains `CONVERSATION_END` or the previous session is >10 minutes stale.

Dynamic columns (master only): master generates ~32 symptom columns on `conversation` via a `setattr` loop over `symptoms.py` at import time; production dropped both the loop and the columns.

Sources: `backend/recover/db.py` on `production` (d7b85b5) and `master` (2e7e25e), `apis.py`, `migrate_data.py`, `fix_server_sequences.py`, migration files under `migrations/versions/`.